



Universidad Autónoma de Santo Domingo

PRIMADA DE AMERICA / Fundada el 28 de octubre de 1538

FACULTAD DE CIENCIAS

ESCUELA DE INFORMÁTICA

INGENIERÍA DE SOFTWARE II (INF-5250 Z05)

Prof. José Ariel Pereyra

Plan de Aseguramiento de la Calidad del Software

Sistema de Gestión de Colegio

Grupo

#1

Integrantes:

- Carol Esther Araujo Rodríguez - 100597534
- Nayeli Alexandra Pacheco Lora - 100632915

Índice

Contenido

1. Introducción	2
2. Objetivos del SQA	2
3. Alcance del SQA.....	2
4. Estándares y Criterios de Calidad	3
4.1 Estándares de Código	3
4.2 Estándares de Base de Datos.....	3
4.3 Estándares de Control de Versiones.....	4
5.1 Revisiones de Código.....	4
5.2 Revisiones de Requerimientos.....	5
5.3 Auditorías de Versionado	5
5.4 Pruebas de Regresión	5
5.5 Verificación de Requerimientos No Funcionales.....	5
7. Herramientas de Aseguramiento de la Calidad	6
8. Lista de Verificación de Calidad por Módulo	7
9. Registro de Calidad por Módulo.....	7
10. Cronograma de Actividades SQA.....	8
11. Conclusión.....	8

1. Introducción

El presente Plan de Aseguramiento de la Calidad del Software (SQA — Software Quality Assurance) describe las actividades, estándares, procedimientos y responsabilidades que el Grupo #1 seguirá para garantizar que el Sistema de Gestión de Colegio cumple con los niveles de calidad requeridos en cada fase de su desarrollo.

El aseguramiento de la calidad no se limita a las pruebas finales del sistema, sino que es un proceso continuo que abarca desde la planificación hasta la entrega del producto. Este plan está alineado con el Plan de Proyecto, la Documentación de Requerimientos, el Plan de Prueba y el Plan de Mediciones elaborados previamente.

2. Objetivos del SQA

- Garantizar que el sistema cumple con todos los requerimientos funcionales y no funcionales definidos en la Documentación de Requerimientos.
- Establecer procesos y estándares de desarrollo que prevengan defectos desde el inicio.
- Verificar que el código fuente es mantenible, legible y sigue buenas prácticas de programación.
- Asegurar que los artefactos del proyecto (documentación, código, base de datos) están versionados correctamente en GitHub.
- Garantizar la seguridad de la información mediante el control de acceso por roles y el cifrado de contraseñas.
- Asegurar que el sistema es usable, está en español y puede ser utilizado sin capacitación técnica avanzada.

3. Alcance del SQA

El Plan de Aseguramiento de la Calidad aplica a todos los componentes del Sistema de Gestión de Colegio, incluyendo:

Componente	Aplica SQA	Descripción
Código fuente (Next.js / React)	☒ Sí	Revisión de estructura, buenas prácticas y mantenibilidad.
Base de datos (MySQL / Prisma)	☒ Sí	Validación de schema, integridad referencial y migraciones.

APIs (Route Handlers)	☒ Sí	Validación de entradas, manejo de errores y respuestas correctas.
Autenticación y autorización	☒ Sí	Control de acceso por roles, sesiones seguras y cifrado.
Interfaz de usuario	☒ Sí	Usabilidad, idioma español y expansividad.
Documentación del proyecto	☒ Sí	Compleitud, coherencia y versionado en GitHub.
Control de versiones (Git/GitHub)	☒ Sí	Commits descriptivos, ramas organizadas y código actualizado.

4. Estándares y Criterios de Calidad

4.1 Estándares de Código

Estándar	Descripción	Herramienta de Verificación
Nombrado en español	Las variables, funciones y componentes deben nombrarse en español para mantener coherencia con el idioma del sistema.	Revisión manual del código
Separación de responsabilidades	Cada archivo debe tener una sola responsabilidad (página, API, componente, utilidad).	Revisión de arquitectura
Manejo de errores	Todas las APIs deben manejar errores con try/catch y devolver mensajes descriptivos.	Revisión de código y pruebas
Validación de datos	Los datos deben validarse tanto en el frontend como en el backend antes de procesarse.	Pruebas con datos inválidos
TypeScript estricto	El código debe compilar sin errores de TypeScript.	Compilador TypeScript / Next.js
Variables de entorno	Las credenciales y configuraciones sensibles deben estar en archivos. env, no en el código.	Revisión del repositorio GitHub

4.2 Estándares de Base de Datos

Estándar	Descripción
Integridad referencial	Todas las relaciones entre tablas deben tener claves foráneas definidas.
Nombres en español	Las tablas y columnas deben nombrarse en español con notación camelCase.

Campos obligatorios	Los campos requeridos deben marcarse como NOT NULL en el schema.
Valores únicos	Los campos como email, cédula y código deben tener restricción UNIQUE.
Contraseñas encriptadas	Las contraseñas deben almacenarse encriptadas con bcrypt (salt rounds: 12).
Migraciones versionadas	Todos los cambios al schema deben realizarse mediante migraciones de Prisma.

4.3 Estándares de Control de Versiones

Estándar	Descripción
Mensajes de commit descriptivos	Los commits deben seguir el formato: tipo: descripción (ej: feat: módulo de autenticación).
Tipos de commit permitidos	feat (nueva función), fix (corrección), docs (documentación), refactor (mejora de código).
Frecuencia mínima de commits	Al menos 1 commit por semana por integrante del equipo.
Archivos excluidos	El archivo .gitignore debe excluir: node_modules/, .env, .env.local, .next/
Ramas organizadas	La rama main solo debe contener código estable y probado.

5. Actividades de Aseguramiento de la Calidad

5.1 Revisiones de Código

Antes de integrar nuevo código a la rama principal, un integrante del equipo diferente al autor debe revisar el código verificando:

- Que el código compila sin errores de TypeScript.
- Que las APIs manejan correctamente los errores con try/catch.
- Que los datos se validan antes de guardarse en la base de datos.
- Que las contraseñas y datos sensibles no están escritos directamente en el código.
- Que los mensajes al usuario están en español y son claros.

5.2 Revisiones de Requerimientos

Al completar cada módulo, el equipo verifica que se cumplen los requerimientos funcionales correspondientes:

Módulo	Requerimientos verificados	Estado
Autenticación	RF-001 (acceso público), RNF-003 (seguridad y cifrado)	☒
Gestión de Usuarios	RF-004 al RF-009 (CRUD, roles, búsqueda)	☒
Módulo Académico	RF-010 al RF-018 (horarios, cursos, calificaciones)	En progreso
Módulo Financiero	RF-019 al RF-026 (pagos, inventario, morosidad)	En progreso
Transporte	RF-027 al RF-030 (rutas, asignaciones)	En progreso
Comunicaciones	RF-030 al RF-033 (blog, notificaciones)	En progreso
Administración	RF-034 al RF-036 (reinicio, reportes)	En progreso

5.3 Auditorías de Versionado

Mensualmente se realizará una auditoría del repositorio GitHub para verificar:

- Que todos los artefactos están actualizados (docs/, src/, prisma/, database/).
- Que el .gitignore excluye los archivos sensibles.
- Que los mensajes de commit siguen el estándar definido.
- Que no hay credenciales expuestas en el código.

5.4 Pruebas de Regresión

Cada vez que se corrija un defecto o se agregue una nueva funcionalidad, se ejecutarán pruebas de regresión para verificar que los módulos anteriores sigan funcionando correctamente.

5.5 Verificación de Requerimientos No Funcionales

RNF	Categoría	Verificación	Frecuencia
RNF-001	Usabilidad	Revisar que toda la interfaz está en español y es intuitiva.	Por módulo completado

RNF-002	Rendimiento	Medir tiempo de respuesta con DevTools del navegador.	Por módulo completado
RNF-003	Seguridad	Verificar control de acceso por roles y encriptación de contraseñas.	Por módulo completado
RNF-004	Disponibilidad	Verificar que el sistema corre sin caídas durante las sesiones de prueba.	Durante pruebas UAT
RNF-005	Confiabilidad	Verificar que los datos se guardan correctamente en MySQL.	Por módulo completado
RNF-006	Escalabilidad	Verificar que nuevos módulos pueden agregarse sin afectar los existentes.	Al agregar cada módulo
RNF-007	Responsividad	Probar en computadora, tableta y teléfono.	Al finalizar el sistema
RNF-008	Privacidad	Verificar que cada rol solo ve su propia información.	Por rol del sistema

6. Roles y Responsabilidades del SQA

Integrante	Matrícula	Responsabilidades SQA
Carol Esther Araujo Rodríguez	100597534	Revisión de módulos financieros y de inventario. Verificación de estándares de código. Actualización de documentación.
Nayeli Alexandra Pacheco Lora	100632915	Líder de calidad. Auditorías de versionado GitHub. Verificación de control de acceso por roles. Coordinación de pruebas UAT.
José A. Minaya	100654987	Revisión de APIs y base de datos. Verificación de rendimiento. Integración y pruebas del módulo académico y transporte.

7. Herramientas de Aseguramiento de la Calidad

Herramienta	Uso en el SQA
TypeScript	Detección de errores de tipo en tiempo de compilación.
ESLint	Análisis estático del código para detectar problemas de estilo y errores comunes.
Next.js Build	Verificación de que el proyecto compila correctamente antes de cada entrega.
MySQL Workbench	Verificación de la integridad de la base de datos y las migraciones.
Prisma Studio	Inspección visual de los datos almacenados en la base de datos.
Chrome DevTools	Medición del tiempo de respuesta y detección de errores en el frontend.

GitHub	Control de versiones, historial de cambios y auditoría de commits.
Postman / DevTools Network	Prueba manual de las APIs y verificación de respuestas HTTP.

8. Lista de Verificación de Calidad por Módulo

Antes de dar por completado cada módulo, el equipo debe verificar los siguientes puntos:

#	Criterio de Verificación	Responsable
1	El módulo compila sin errores de TypeScript.	Desarrollador
2	Todas las APIs tienen manejo de errores con try/catch.	Desarrollador
3	Los datos se validan en el frontend y en el backend.	Desarrollador
4	Los mensajes al usuario están en español.	Revisor
5	Solo los roles autorizados pueden acceder al módulo.	Nayeli
6	Los datos se guardan correctamente en MySQL.	José
7	El tiempo de respuesta es menor a 3 segundos.	José
8	Los casos de prueba del módulo pasan exitosamente.	Todo el equipo
9	El código fue revisado por otro integrante del equipo.	Revisor
10	Los cambios fueron subidos a GitHub con commit descriptivo.	Nayeli
11	La documentación del módulo está actualizada.	Carol
12	El módulo funciona correctamente en móvil y computadora.	Carol

9. Registro de Calidad por Módulo

Módulo	Compilación	Pruebas	Acceso por Rol	Rendimiento	Versionado	Calidad General
Autenticación	☒	☒	☒	☒	☒	☒ Aprobado
Gestión de Usuarios	☒	☒	☒	☒	☒	☒ Aprobado
Módulo Académico	☒	☐	☒	☒	☒	En revisión
Módulo Financiero	☒	☐	☒	☒	☒	En revisión
Inventario Uniformes	☒	☒	☒	☒	☒	☒ Aprobado
Transporte Escolar	☒	☐	☒	☐	☐	En progreso

Administración	☒	☒	☒	☒	☒	☒ Aprobado
Portal Público	☒	☐	☒	☒	☒	☐ En revisión

10. Cronograma de Actividades SQA

Actividad SQA	Semanas	Responsable
Revisión de requerimientos por módulo	4–12	Todo el equipo
Revisión de código antes de cada commit	4–15	Revisor designado
Auditoría de versionado GitHub	8 y 12	Nayeli
Verificación de RNF por módulo	6–13	José
Pruebas de regresión	13–14	Todo el equipo
Pruebas de usuario UAT	14	Todo el equipo
Corrección de defectos críticos	14–15	Desarrollador responsable
Verificación final de lista de calidad	15	Todo el equipo
Certificación de calidad para presentación	15–16	Nayeli (líder)

11. Conclusión

El Plan de Aseguramiento de la Calidad del Software establece un marco claro de actividades, estándares y responsabilidades que el equipo seguirá para garantizar que el Sistema de Gestión de Colegio sea un producto de alta calidad.

La aplicación de este plan desde las primeras fases del desarrollo ha permitido identificar y corregir defectos oportunamente, mantener el código organizado y garantizar que el sistema cumple con los requerimientos funcionales y no funcionales establecidos. El equipo continuará aplicando estas prácticas hasta la presentación final en la Semana 16.